

AI SOFTWARE DEVELOPMENT INTERNSHIP

EMAN MASOOD (BSSE)

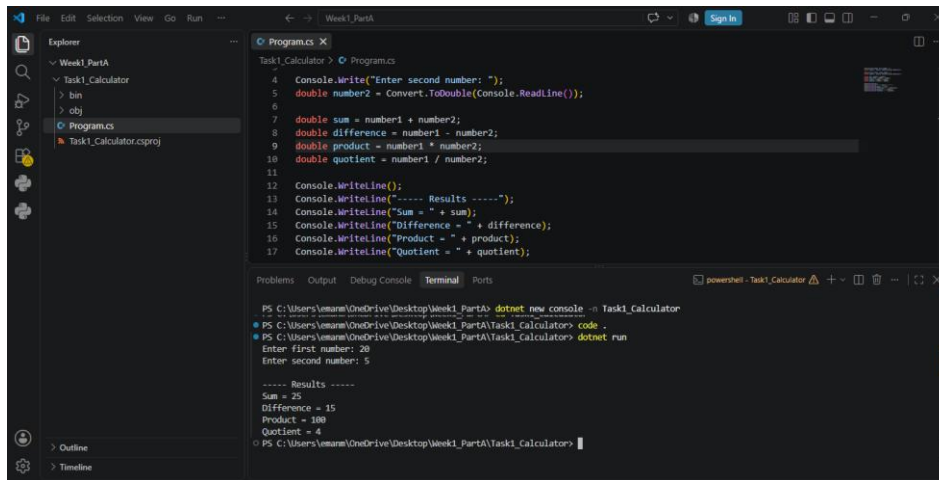
WEEK 1 - Programming Foundations - C#/.NET +
Angular/TypeScript

PART A - C# AND .NET BASICS

Small exercises

1. Write a program that asks for two numbers and prints their sum, difference, product, and quotient

Output:



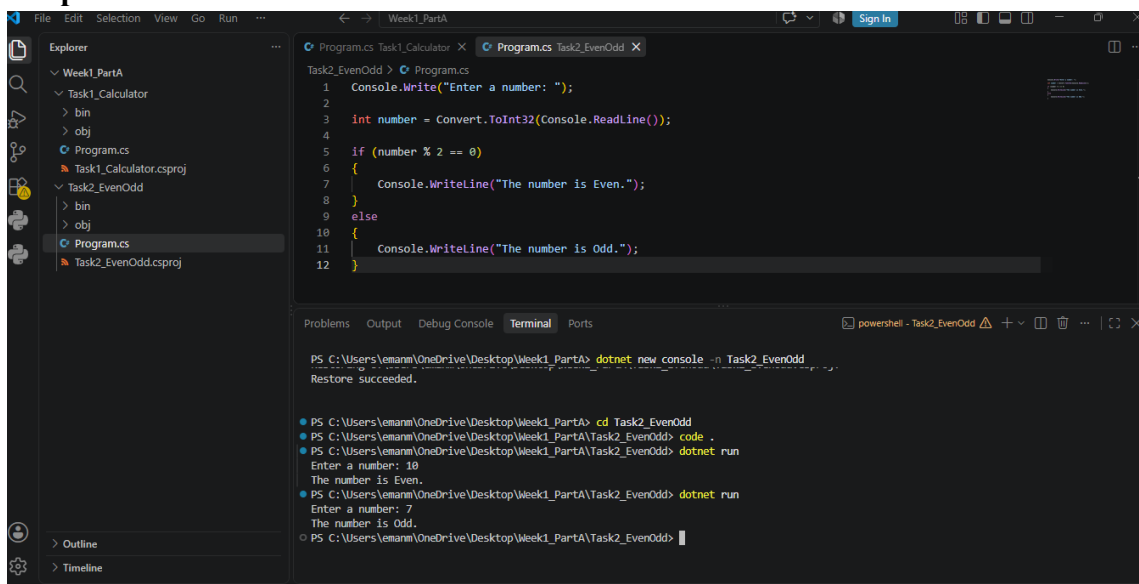
```
Task1_Calculator > Program.cs
4 Console.WriteLine("Enter second number: ");
5 double number2 = Convert.ToDouble(Console.ReadLine());
6
7 double sum = number1 + number2;
8 double difference = number1 - number2;
9 double product = number1 * number2;
10 double quotient = number1 / number2;
11
12 Console.WriteLine();
13 Console.WriteLine("----- Results -----");
14 Console.WriteLine("Sum = " + sum);
15 Console.WriteLine("Difference = " + difference);
16 Console.WriteLine("Product = " + product);
17 Console.WriteLine("Quotient = " + quotient);

Problems Output Debug Console Terminal Ports
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA> dotnet new console -n Task1_Calculator
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA\Task1_Calculator> code .
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA\Task1_Calculator> dotnet run
Enter first number: 20
Enter second number: 5

----- Results -----
Sum = 25
Difference = 15
Product = 100
Quotient = 4
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA\Task1_Calculator>
```

2. Write a program that determines whether a number is even or odd

Output:



```
Task2_EvenOdd > Program.cs
1 Console.WriteLine("Enter a number: ");
2
3 int number = Convert.ToInt32(Console.ReadLine());
4
5 if (number % 2 == 0)
6 {
7     Console.WriteLine("The number is Even.");
8 }
9 else
10 {
11     Console.WriteLine("The number is Odd.");
12 }

Problems Output Debug Console Terminal Ports
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA> dotnet new console -n Task2_EvenOdd
Restore succeeded.

PS C:\Users\emam\OneDrive\Desktop\Week1_PartA> cd Task2_EvenOdd
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA\Task2_EvenOdd> code .
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA\Task2_EvenOdd> dotnet run
Enter a number: 18
The number is Even.
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA\Task2_EvenOdd> dotnet run
Enter a number: 7
The number is Odd.
PS C:\Users\emam\OneDrive\Desktop\Week1_PartA\Task2_EvenOdd>
```

3. Write a program that prints numbers 1-100 and replaces multiples of 3 with 'Fizz' and multiples of 5 with 'Buzz'

Output:

```
Task3_FizzBuzz > Program.cs
1  for (int i = 1; i <= 100; i++)
2  {
3      if (i % 3 == 0 && i % 5 == 0)
4      {
5          Console.WriteLine("FizzBuzz");
6      }
7      Console.WriteLine(i);
8  }
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA> cd Task3_FizzBuzz
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task3_FizzBuzz> code .
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task3_FizzBuzz> dotnet run
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
```

```
Task3_FizzBuzz > Program.cs
1  for (int i = 1; i <= 100; i++)
2  {
3      if (i % 3 == 0 && i % 5 == 0)
4      {
5          Console.WriteLine("FizzBuzz");
6      }
7      Console.WriteLine(i);
8  }
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task3_FizzBuzz> dotnet run
44
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
Fizz
52
53
Fizz
Buzz
56
Fizz
58
59
FizzBuzz
61
62
Fizz
64
```

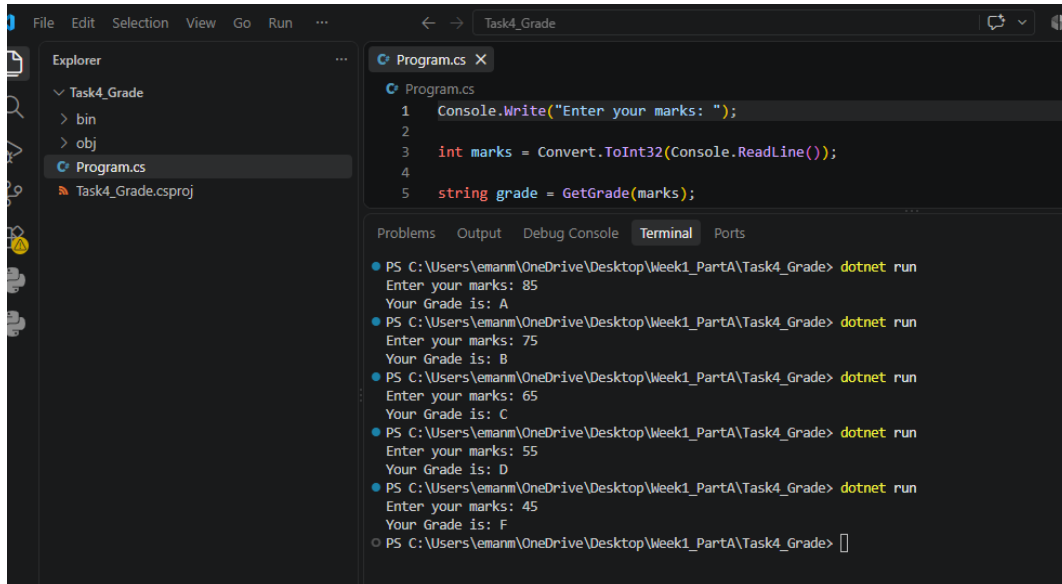
```
Task3_FizzBuzz > Program.cs
1  for (int i = 1; i <= 100; i++)
2  {
3      if (i % 3 == 0 && i % 5 == 0)
4      {
5          Console.WriteLine("FizzBuzz");
6      }
7      Console.WriteLine(i);
8  }
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task3_FizzBuzz> dotnet run
49
Buzz
Fizz
82
83
Fizz
Buzz
86
Fizz
88
89
FizzBuzz
91
92
Fizz
94
Buzz
Fizz
97
98
Fizz
Buzz
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task3_FizzBuzz>
```

4. Write a method that receives marks and returns a grade.

Output:



The screenshot shows the Visual Studio IDE with a project named 'Task4_Grade'. The 'Program.cs' file contains the following code:

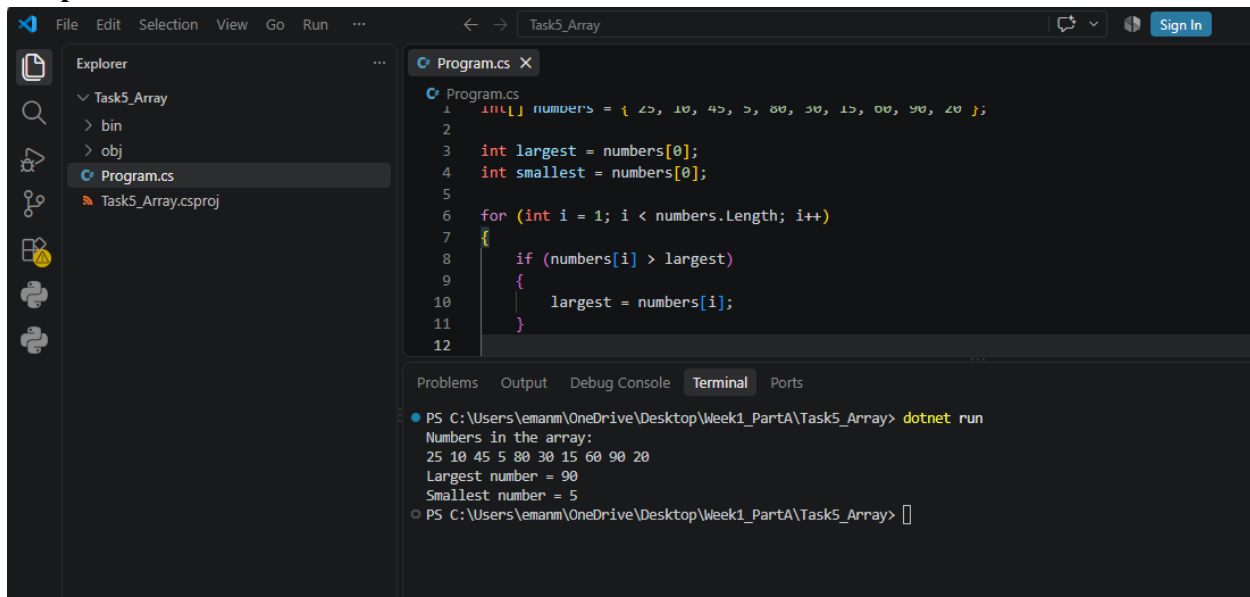
```
1 Console.WriteLine("Enter your marks: ");
2
3 int marks = Convert.ToInt32(Console.ReadLine());
4
5 string grade = GetGrade(marks);
```

The 'Terminal' window shows the output of running the program multiple times:

```
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task4_Grade> dotnet run
Enter your marks: 85
Your Grade is: A
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task4_Grade> dotnet run
Enter your marks: 75
Your Grade is: B
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task4_Grade> dotnet run
Enter your marks: 65
Your Grade is: C
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task4_Grade> dotnet run
Enter your marks: 55
Your Grade is: D
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task4_Grade> dotnet run
Enter your marks: 45
Your Grade is: F
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task4_Grade> 
```

5. Create an array of 10 numbers and find the largest and smallest value.

Output:



The screenshot shows the Visual Studio IDE with a project named 'Task5_Array'. The 'Program.cs' file contains the following code:

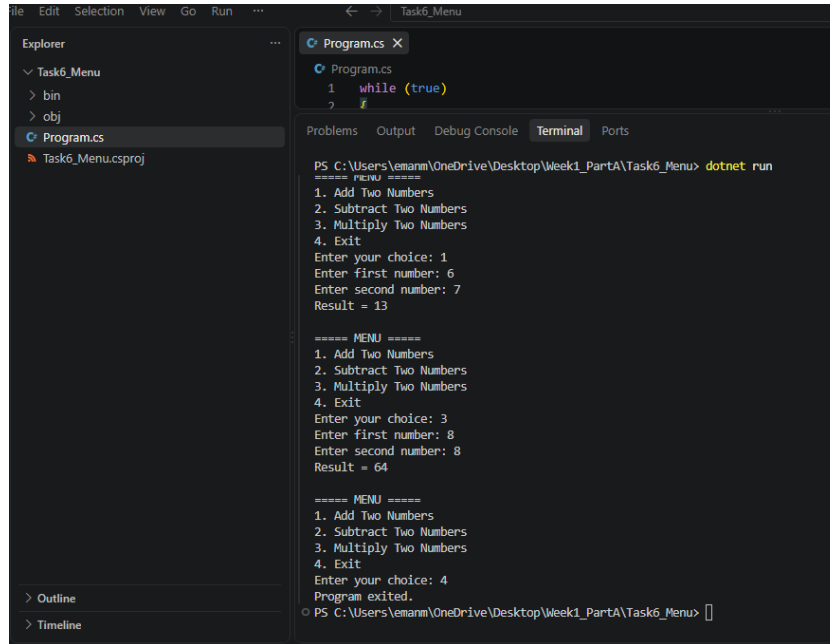
```
1 int[] numbers = { 25, 10, 45, 5, 80, 30, 15, 00, 90, 20 };
2
3 int largest = numbers[0];
4 int smallest = numbers[0];
5
6 for (int i = 1; i < numbers.Length; i++)
7 {
8     if (numbers[i] > largest)
9     {
10         largest = numbers[i];
11     }
12 }
```

The 'Terminal' window shows the output of running the program:

```
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task5_Array> dotnet run
Numbers in the array:
25 10 45 5 80 30 15 60 90 20
Largest number = 90
Smallest number = 5
PS C:\Users\emamm\OneDrive\Desktop\Week1_PartA\Task5_Array> 
```

6. Write a menu-driven console program with an Exit option

Output:



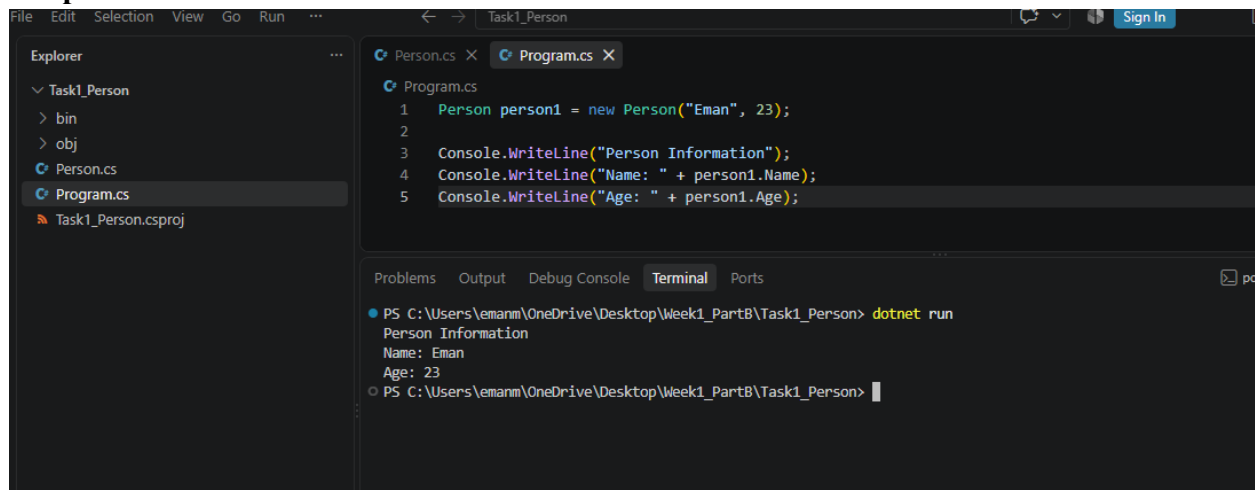
```
file Edit Selection View Go Run ... Task6_Menu
Explorer
  Task6_Menu
    bin
    obj
    Program.cs
    Task6_Menu.csproj
  Outline
  Timeline
  Program.cs
    1 while (true)
    2 {
    3     ===== MENU =====
    4     1. Add Two Numbers
    5     2. Subtract Two Numbers
    6     3. Multiply Two Numbers
    7     4. Exit
    8     Enter your choice: 1
    9     Enter first number: 6
   10     Enter second number: 7
   11     Result = 13
   12
   13     ===== MENU =====
   14     1. Add Two Numbers
   15     2. Subtract Two Numbers
   16     3. Multiply Two Numbers
   17     4. Exit
   18     Enter your choice: 3
   19     Enter first number: 8
   20     Enter second number: 8
   21     Result = 64
   22
   23     ===== MENU =====
   24     1. Add Two Numbers
   25     2. Subtract Two Numbers
   26     3. Multiply Two Numbers
   27     4. Exit
   28     Enter your choice: 4
   29     Program exited.
   30 }
  Problems Output Debug Console Terminal Ports
  PS C:\Users\emanm\OneDrive\Desktop\Week1_PartA\Task6_Menu> dotnet run
  ===== MENU =====
  1. Add Two Numbers
  2. Subtract Two Numbers
  3. Multiply Two Numbers
  4. Exit
  Enter your choice: 1
  Enter first number: 6
  Enter second number: 7
  Result = 13
  ===== MENU =====
  1. Add Two Numbers
  2. Subtract Two Numbers
  3. Multiply Two Numbers
  4. Exit
  Enter your choice: 3
  Enter first number: 8
  Enter second number: 8
  Result = 64
  ===== MENU =====
  1. Add Two Numbers
  2. Subtract Two Numbers
  3. Multiply Two Numbers
  4. Exit
  Enter your choice: 4
  Program exited.
  PS C:\Users\emanm\OneDrive\Desktop\Week1_PartA\Task6_Menu>
```

PART B - OBJECT-ORIENTED C#

Practice

1. Create a Person class with Name and Age

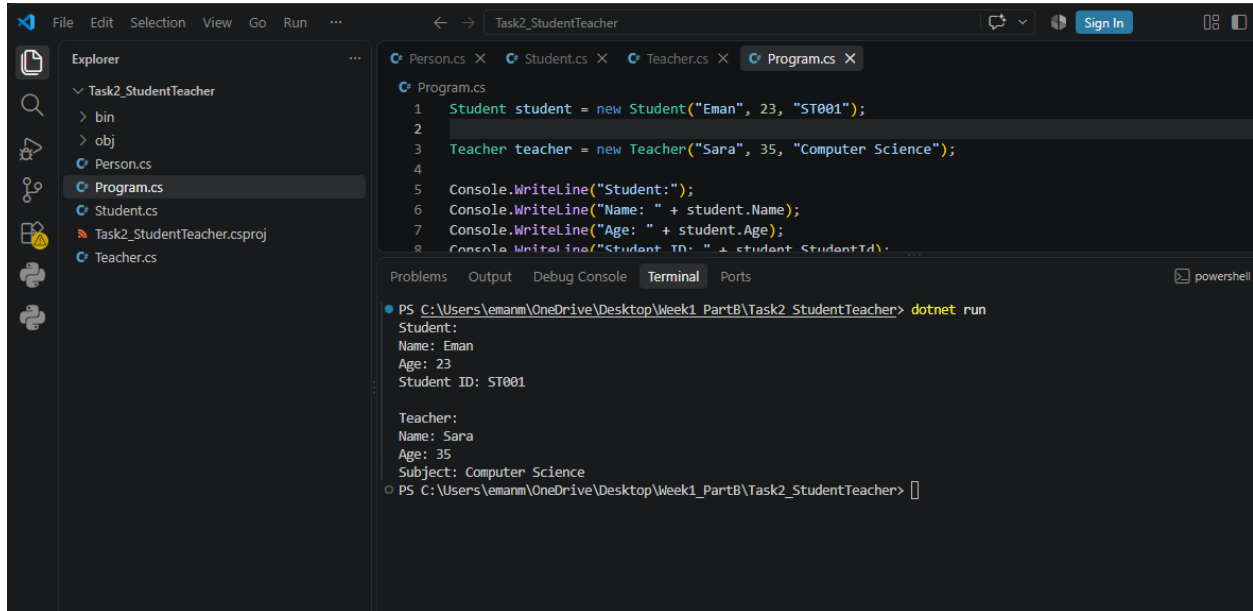
Output:



```
File Edit Selection View Go Run ... Task1_Person
Explorer
  Task1_Person
    bin
    obj
    Person.cs
    Program.cs
    Task1_Person.csproj
  Outline
  Timeline
  Person.cs
  Program.cs
    1 Person person1 = new Person("Eman", 23);
    2
    3 Console.WriteLine("Person Information");
    4 Console.WriteLine("Name: " + person1.Name);
    5 Console.WriteLine("Age: " + person1.Age);
  Problems Output Debug Console Terminal Ports
  PS C:\Users\emanm\OneDrive\Desktop\Week1_PartB\Task1_Person> dotnet run
  Person Information
  Name: Eman
  Age: 23
  PS C:\Users\emanm\OneDrive\Desktop\Week1_PartB\Task1_Person>
```

2. Create Student and Teacher classes and decide what should be inherited

Output:



The screenshot shows the Visual Studio IDE with a project named 'Task2_StudentTeacher'. The Explorer pane on the left shows the project structure with files: bin, obj, Person.cs, Program.cs, Student.cs, Task2_StudentTeacher.csproj, and Teacher.cs. The Solution Explorer on the right shows the code for Program.cs, which contains the following code:

```
1 Student student = new Student("Eman", 23, "ST001");
2
3 Teacher teacher = new Teacher("Sara", 35, "Computer Science");
4
5 Console.WriteLine("Student:");
6 Console.WriteLine("Name: " + student.Name);
7 Console.WriteLine("Age: " + student.Age);
8 Console.WriteLine("Student ID: " + student.StudentId);
```

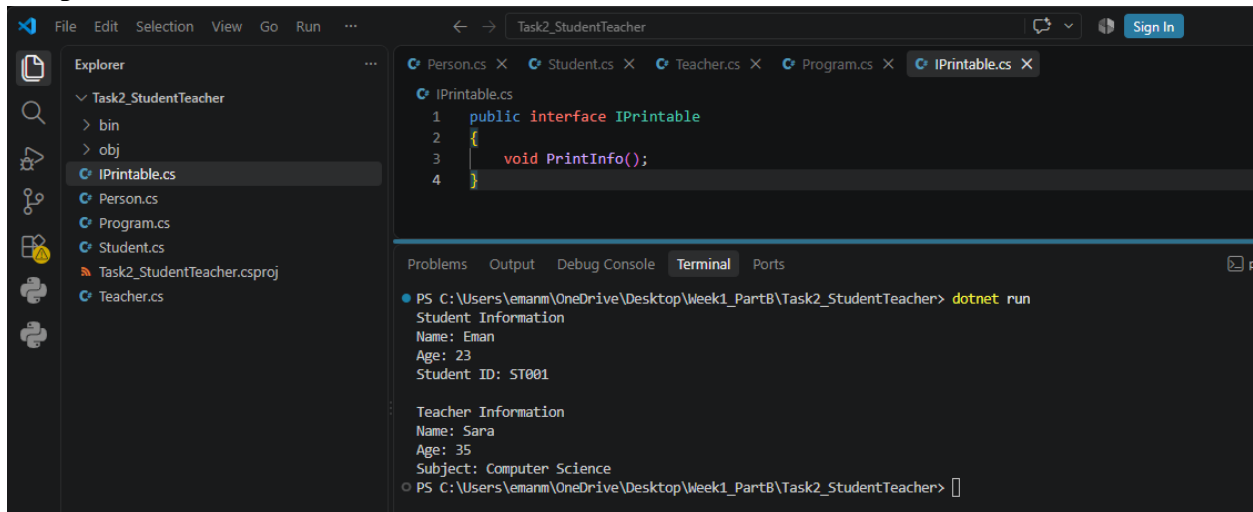
The Output pane at the bottom shows the results of running the application:

```
PS C:\Users\emamm\OneDrive\Desktop\Week1_Part8\Task2_StudentTeacher> dotnet run
Student:
Name: Eman
Age: 23
Student ID: ST001

Teacher:
Name: Sara
Age: 35
Subject: Computer Science
PS C:\Users\emamm\OneDrive\Desktop\Week1_Part8\Task2_StudentTeacher>
```

3. Create an IPrintable interface and implement it

Output:



The screenshot shows the Visual Studio IDE with the same project 'Task2_StudentTeacher'. The Explorer pane on the left shows the project structure with files: bin, obj, IPrintable.cs, Person.cs, Program.cs, Student.cs, Task2_StudentTeacher.csproj, and Teacher.cs. The Solution Explorer on the right shows the code for IPrintable.cs, which contains the following code:

```
1 public interface IPrintable
2 {
3     void PrintInfo();
4 }
```

The Output pane at the bottom shows the results of running the application:

```
PS C:\Users\emamm\OneDrive\Desktop\Week1_Part8\Task2_StudentTeacher> dotnet run
Student Information
Name: Eman
Age: 23
Student ID: ST001

Teacher Information
Name: Sara
Age: 35
Subject: Computer Science
PS C:\Users\emamm\OneDrive\Desktop\Week1_Part8\Task2_StudentTeacher>
```

Task 4: Inheritance vs Composition

Inheritance:

Inheritance is useful when there is an "is-a" relationship between classes. For example, a Student is a Person and a Teacher is a Person. In this case, Student and Teacher can inherit common properties such as Name and Age from Person.

Composition:

Composition is useful when one class has another object. For example, a Car has an Engine. A Car is not an Engine, so inheritance would not make sense. Composition allows the Car class to use an Engine object without inheriting from it.

Conclusion:

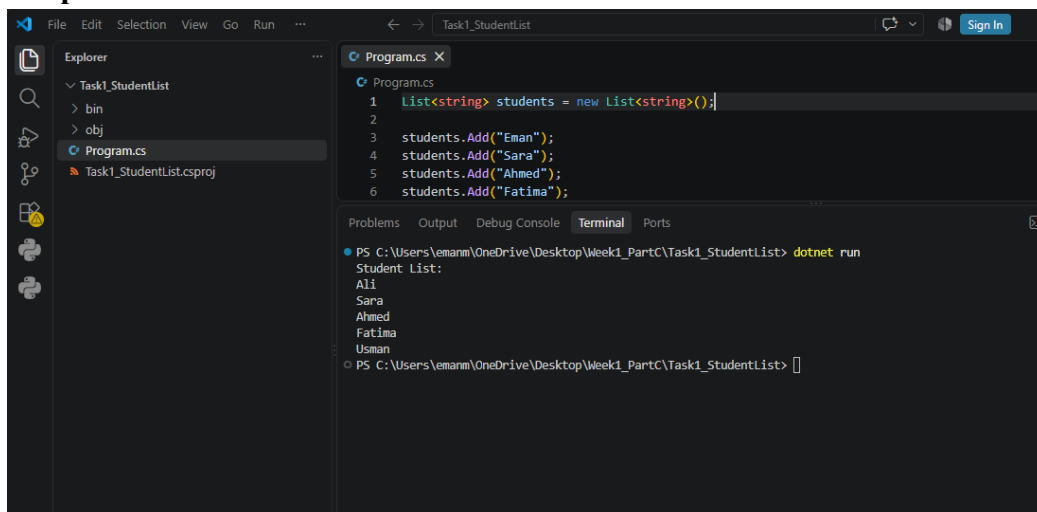
I would use inheritance when there is a clear "is-a" relationship. I would use composition when there is a "has-a" relationship.

PART C - COLLECTIONS, EXCEPTIONS, AND LINQ

Practice

1. Create a List and add five students

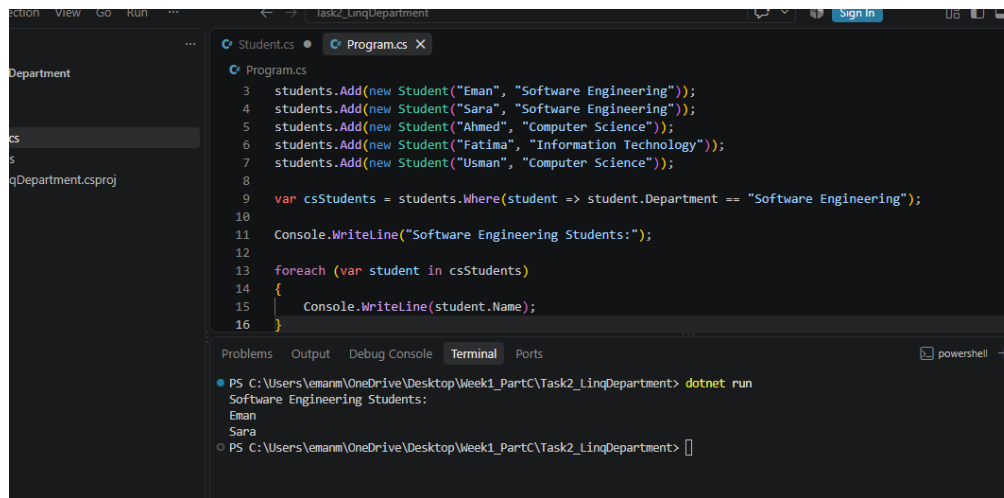
Output:



```
File Edit Selection View Go Run ... Task1_StudentList Sign In
Explorer
  Task1_StudentList
    > bin
    > obj
    Program.cs
    Task1_StudentList.csproj
  Program.cs X
    Program.cs
    1 List<string> students = new List<string>();
    2
    3 students.Add("Eman");
    4 students.Add("Sara");
    5 students.Add("Ahmed");
    6 students.Add("Fatima");
  Problems Output Debug Console Terminal Ports
  PS C:\Users\emarm\OneDrive\Desktop\Week1_PartC\Task1_StudentList> dotnet run
  Student List:
  Ali
  Sara
  Ahmed
  Fatima
  Usman
  PS C:\Users\emarm\OneDrive\Desktop\Week1_PartC\Task1_StudentList> []
```

2. Find students from a particular department using LINQ

Output:



The screenshot shows a Visual Studio IDE with a C# project named 'Task2_LinqDepartment'. The 'Program.cs' file contains the following code:

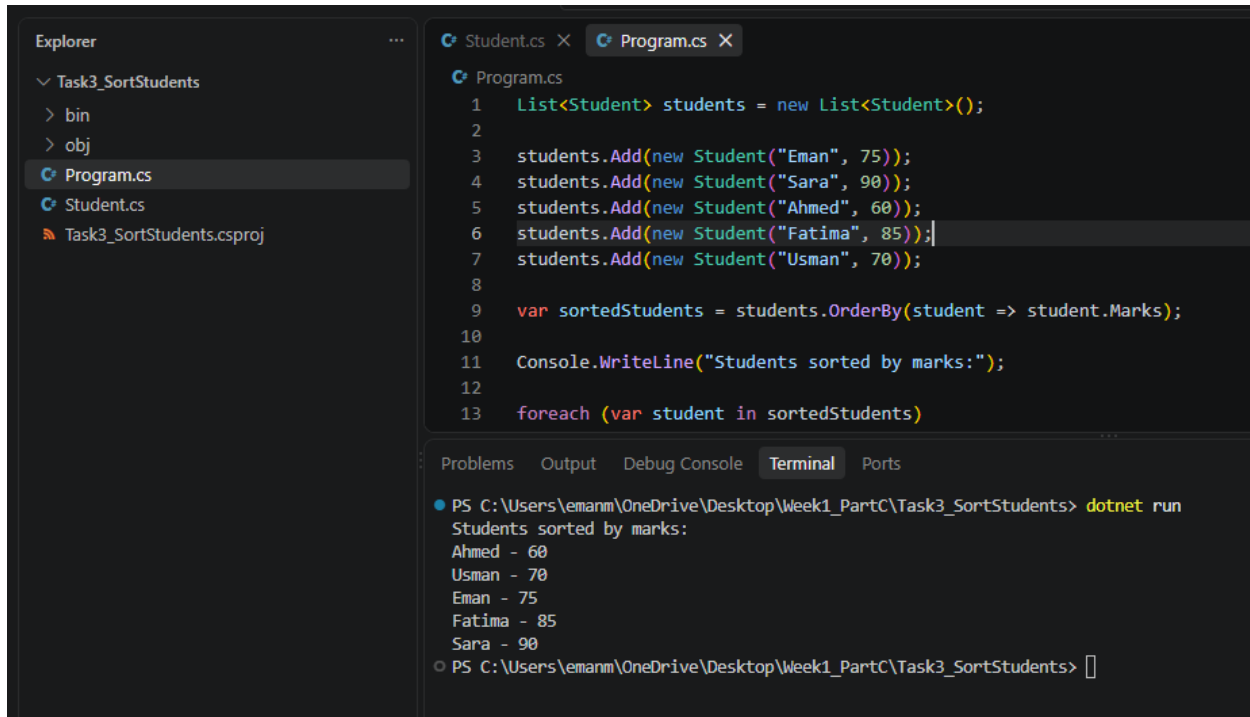
```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  namespace Task2_LinqDepartment
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             List<Student> students = new List<Student>();
11             students.Add(new Student("Eman", "Software Engineering"));
12             students.Add(new Student("Sara", "Software Engineering"));
13             students.Add(new Student("Ahmed", "Computer Science"));
14             students.Add(new Student("Fatima", "Information Technology"));
15             students.Add(new Student("Usman", "Computer Science"));
16
17             var csStudents = students.Where(student => student.Department == "Software Engineering");
18
19             Console.WriteLine("Software Engineering Students:");
20
21             foreach (var student in csStudents)
22             {
23                 Console.WriteLine(student.Name);
24             }
25         }
26     }
27 }
```

The terminal output shows the execution of the program:

```
PS C:\Users\emam\OneDrive\Desktop\Week1_PartC\Task2_LinqDepartment> dotnet run
Software Engineering Students:
Eman
Sara
PS C:\Users\emam\OneDrive\Desktop\Week1_PartC\Task2_LinqDepartment>
```

3. Sort students by marks.

Output:



The screenshot shows a Visual Studio IDE with a C# project named 'Task3_SortStudents'. The 'Program.cs' file contains the following code:

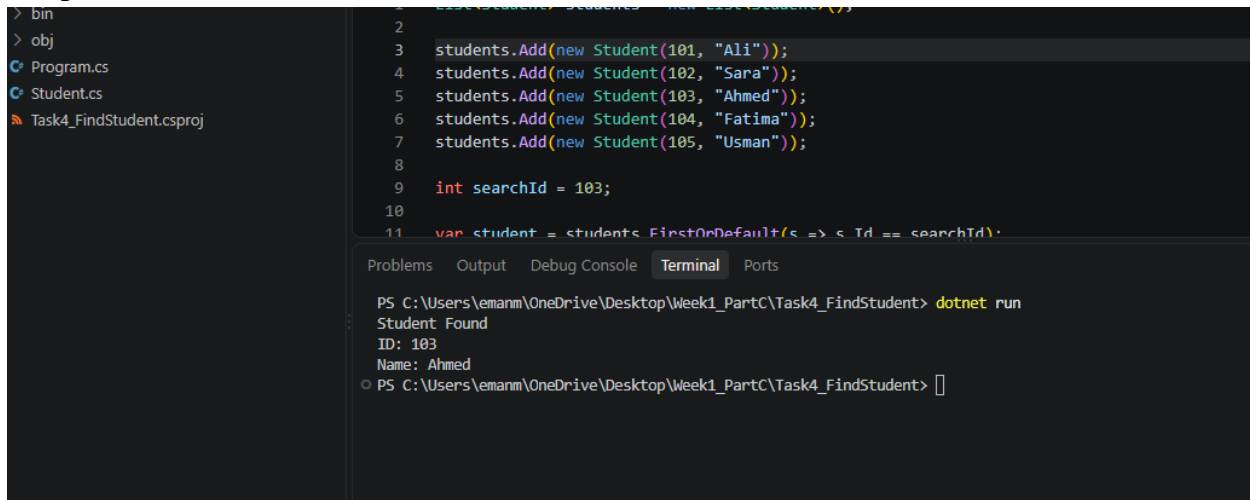
```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  namespace Task3_SortStudents
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             List<Student> students = new List<Student>();
11             students.Add(new Student("Eman", 75));
12             students.Add(new Student("Sara", 90));
13             students.Add(new Student("Ahmed", 60));
14             students.Add(new Student("Fatima", 85));
15             students.Add(new Student("Usman", 70));
16
17             var sortedStudents = students.OrderBy(student => student.Marks);
18
19             Console.WriteLine("Students sorted by marks:");
20
21             foreach (var student in sortedStudents)
22             {
23                 Console.WriteLine($"{student.Name} - {student.Marks}");
24             }
25         }
26     }
27 }
```

The terminal output shows the execution of the program:

```
PS C:\Users\emam\OneDrive\Desktop\Week1_PartC\Task3_SortStudents> dotnet run
Students sorted by marks:
Ahmed - 60
Usman - 70
Eman - 75
Fatima - 85
Sara - 90
PS C:\Users\emam\OneDrive\Desktop\Week1_PartC\Task3_SortStudents>
```

4. Find a student by ID.

Output:



```
1 class Student {
2     public int Id { get; set; }
3     public string Name { get; set; }
4 }
5
6 List<Student> students = new List<Student>();
7
8 students.Add(new Student(101, "Ali"));
9 students.Add(new Student(102, "Sara"));
10 students.Add(new Student(103, "Ahmed"));
11 students.Add(new Student(104, "Fatima"));
12 students.Add(new Student(105, "Usman"));
13
14 int searchId = 103;
15
16 var student = students.FirstOrDefault(s => s.Id == searchId);
17
18 if (student != null)
19 {
20     Console.WriteLine("Student Found");
21     Console.WriteLine($"ID: {student.Id}");
22     Console.WriteLine($"Name: {student.Name}");
23 }
24 else
25 {
26     Console.WriteLine("Student not found");
27 }
```

PS C:\Users\emanm\OneDrive\Desktop\Week1_PartC\Task4_FindStudent> dotnet run

Student Found

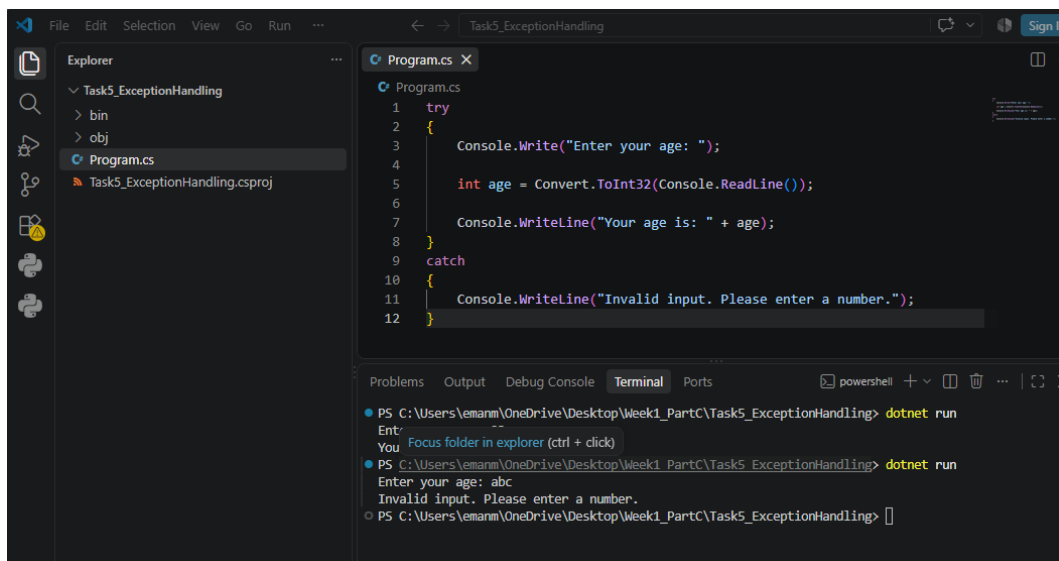
ID: 103

Name: Ahmed

PS C:\Users\emanm\OneDrive\Desktop\Week1_PartC\Task4_FindStudent>

5. Handle invalid numeric input without crashing.

Output:



```
1 try
2 {
3     Console.WriteLine("Enter your age: ");
4
5     int age = Convert.ToInt32(Console.ReadLine());
6
7     Console.WriteLine($"Your age is: " + age);
8 }
9 catch
10 {
11     Console.WriteLine("Invalid input. Please enter a number.");
12 }
```

PS C:\Users\emanm\OneDrive\Desktop\Week1_PartC\Task5_ExceptionHandling> dotnet run

Enter your age: 25

Your age is: 25

PS C:\Users\emanm\OneDrive\Desktop\Week1_PartC\Task5_ExceptionHandling> dotnet run

Enter your age: abc

Invalid input. Please enter a number.

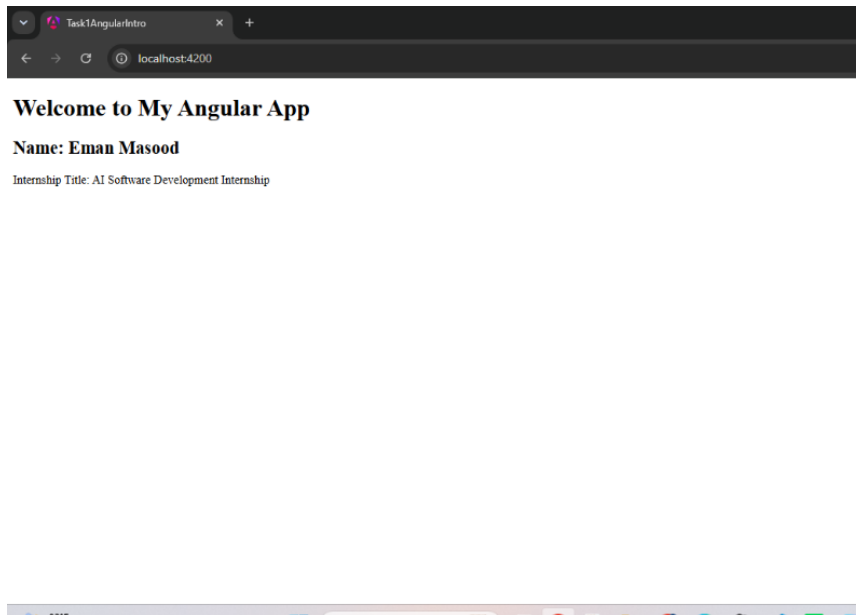
PS C:\Users\emanm\OneDrive\Desktop\Week1_PartC\Task5_ExceptionHandling>

PART D - ANGULAR AND TYPESCRIPT BASIC

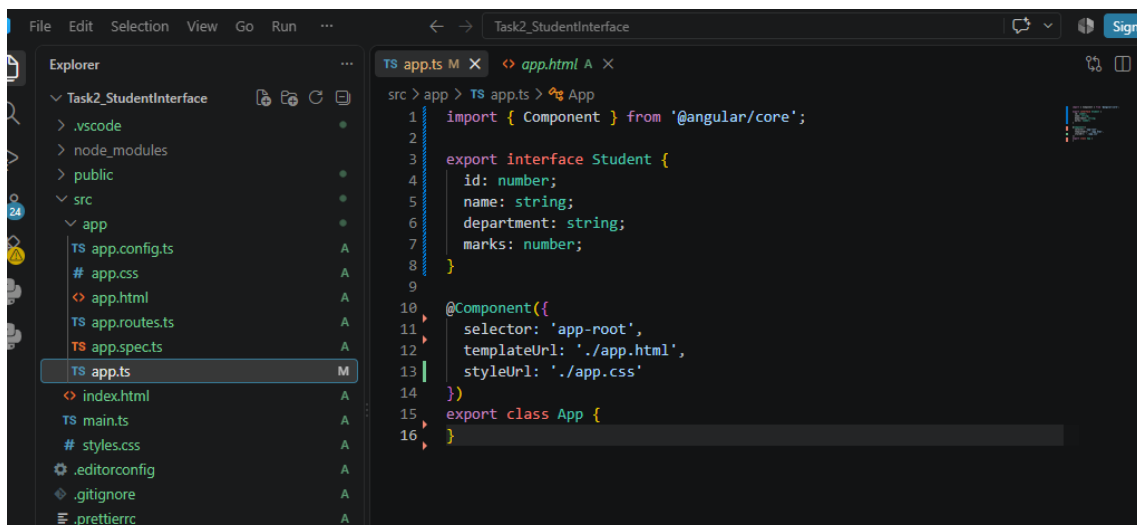
Angular practice

1. Create a simple Angular app that displays your name and internship title

Output:

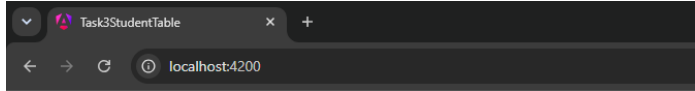


2. Create a Student interface in TypeScript.



3. Create an array of students and display them in a table.

Output:

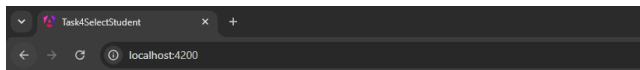


Student List

ID	Name	Department	Marks
101	Eman	Software Engineering	85
102	Sara	Software Engineering	90
103	Ahmed	Computer Science	75
104	Fatima	Information Technology	88
105	Usman	Computer Science	70

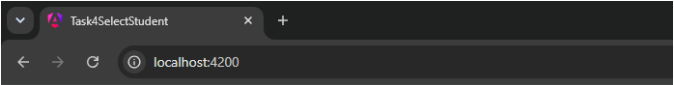
4. Add a button that selects a student and shows their details

Output:



Student List

Name	Action
Eman	Select
Sara	Select
Ahmed	Select
Fatima	Select
Usman	Select



Student List

Name	Action
Eman	Select
Sara	Select
Ahmed	Select
Fatima	Select
Usman	Select

Selected Student

ID: 101
Name: Eman
Department: Software Engineering
Marks: 85



Student List

Name	Action
Eman	Select
Sara	Select
Ahmed	Select
Fatima	Select
Usman	Select

Selected Student

ID: 104
Name: Fatima
Department: Information Technology
Marks: 88

